

AuMgO / cumulene / bio_dimers: checkpoint-parity diagnostics from metawork-54

This notebook visualizes raw diagnostic data generated by a peer Claude session (metawork-54), working in `lorem-tmlr-archive` on the same torch-port-of-lorem-jax effort as this project's

`15_long_range_scan_lorem_jax_ported_torch.ipynb` (via `JaxParityBackbone / JaxParityLongRange`), on **AuMgO** (a periodic slab) and **bio_dimers** (periodic, large box) -- plus **cumulene** (a warm-started fine-tune, snapshotted mid-run).

Update: the periodic (AuMgO, bio_dimers) discrepancy described in sections 1-6 below has since been root-caused and fixed -- see section 7 onward and the updated Takeaways at the end. Sections 1-6 are kept as originally written, as the record of the investigation that found the bug; they no longer describe the current state of the code.

1. AuMgO: stage-by-stage parity (torch port vs jax reference)

For one AuMgO frame (110 atoms), `metawork-54` captured the intermediate output of every module in the forward pass on both the jax reference (`flax.linen`'s `capture_intermediates=True`) and the torch port, and recorded (`torch_value` , `jax_value` , `diff`) for each. Unlike this project's SN2 port (`15_long_range_scan_lorem_jax_ported_torch.ipynb` , agreement $\sim 2e-5$ eV), **this AuMgO port is not yet fully closed** -- the scalar-only stages (`Dense_0` , `Update_0` , `Update_1` , `MLP_0` , `MLP_1`) match near the float32 noise floor, but every tensor/spherical-valued stage (`TensorDense_0` , `TensorDense_1` , `Dense_3` , `Tensor_0` -- orange bars) shows a large elementwise difference, consistent with an e3x component-ordering or CG-phase-convention mismatch (the same class of issue `e3x_compat.py` / `test_e3x_parity.py` exist to handle for the SR path, but evidently not fully resolved for every module here). Interestingly, the *rotationally-invariant* scalar quantities computed from those tensors (`Update_1` , `Update_2`) partially self-correct, and the final `TOTAL` per-atom energy differs by only ~ 0.06 eV -- much better than the raw tensor-stage differences suggest, but still not float32-noise-level like the SN2 case.

```
In [1]: import pickle
import numpy as np
```

```

with open("metawork54_results/intermediate_diagnostics.pkl", "rb") as fh:
    diag = pickle.load(fh)

for stage, (torch_v, jax_v, diff) in diag.items():
    print(f"{stage:25s} max|diff|={np.abs(diff).max():.4e}    max|value|~{max

```

```

Dense_0 (nodes_scalar init) max|diff|=1.5691e-07    max|value|~2.54
Update_0                    max|diff|=2.4639e-05    max|value|~7.54
TensorDense_0               max|diff|=6.1759e+00    max|value|~5.71
Update_1                    max|diff|=2.7766e-05    max|value|~5.90
MLP_0 (SR energy)           max|diff|=2.7335e-06    max|value|~2.70
MLP_1 (scalar charge)       max|diff|=6.2971e-05    max|value|~31.20
TensorDense_1               max|diff|=1.9564e+01    max|value|~19.90
Dense_3                     max|diff|=9.4686e+00    max|value|~14.85
Tensor_0                    max|diff|=1.7525e+01    max|value|~19.77
Update_2                    max|diff|=1.6406e+00    max|value|~4.15
MLP_2 (LR energy)           max|diff|=6.3993e-02    max|value|~0.46
TOTAL                       max|diff|=6.3990e-02    max|value|~3.16

```

```

In [2]: import matplotlib.pyplot as plt
        from matplotlib.patches import Patch

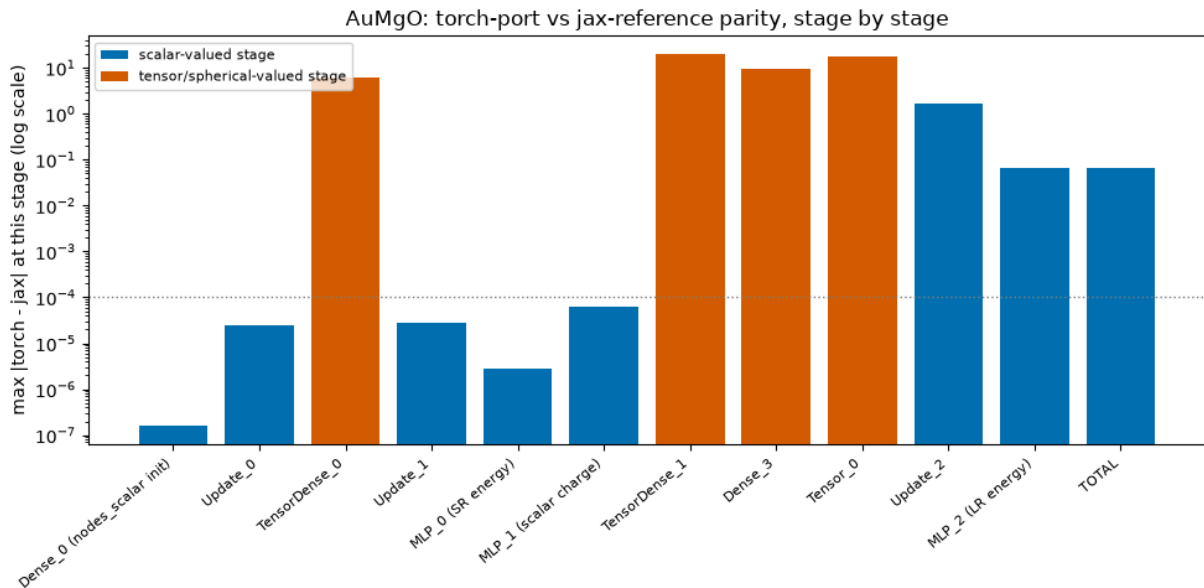
        BLUE, ORANGE = "#0072B2", "#D55E00"
        tensor_stages = {"TensorDense_0", "TensorDense_1", "Dense_3", "Tensor_0"}
        stages = list(diag.keys())
        max_diff = [float(np.abs(diag[s][2]).max()) for s in stages]
        colors = [ORANGE if s in tensor_stages else BLUE for s in stages]

        fig, ax = plt.subplots(figsize=(10, 5))
        ax.bar(range(len(stages)), max_diff, color=colors)
        ax.set_yscale("log")
        ax.set_xticks(range(len(stages)))
        ax.set_xticklabels(stages, rotation=40, ha="right", fontsize=8)
        ax.set_ylabel("max |torch - jax| at this stage (log scale)")
        ax.set_title("AuMg0: torch-port vs jax-reference parity, stage by stage")
        ax.axhline(1e-4, color="0.5", linestyle=":", linewidth=1)
        ax.legend(handles=[Patch(facecolor=BLUE, label="scalar-valued stage"),
                           Patch(facecolor=ORANGE, label="tensor/spherical-valued st
                               fontsize=8, loc="upper left")
        fig.tight_layout()

```

mkdir -p failed for path /home/boittier/.config/matplotlib: [Errno 13] Permission denied: '/home/boittier/.config/matplotlib'

Matplotlib created a temporary cache directory at /tmp/matplotlib-rtfc5l4d because there was an issue with the default path ({configdir}); it is highly recommended to set the MPLCONFIGDIR environment variable to a writable directory, in particular to speed up the import of Matplotlib and to better support multiprocessing.



2. AuMgO: total-energy parity across 20 frames

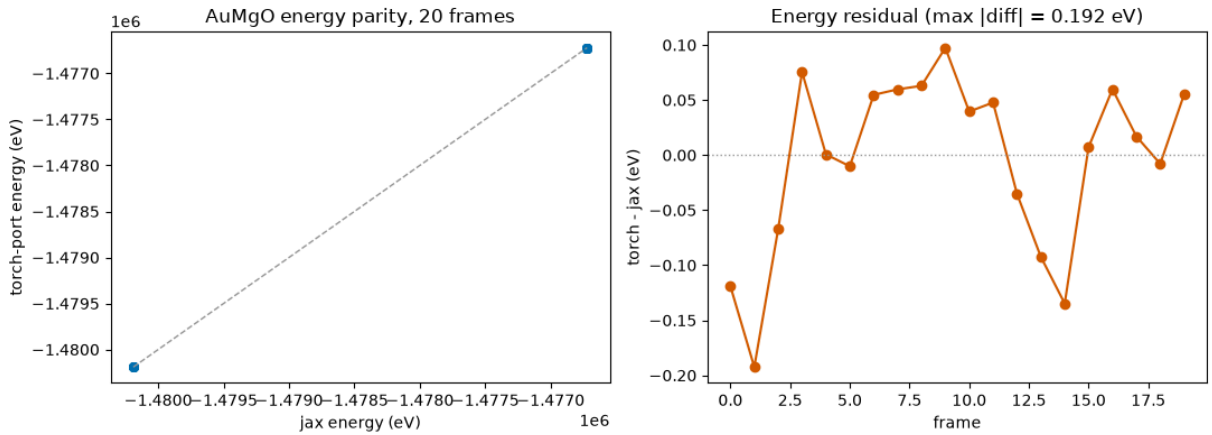
Energies here are ~1.48 million eV (a full periodic slab total, not per-atom) -- so a 0.19 eV max discrepancy is a tiny *relative* error, but on an absolute scale it's far short of the SN2 port's ~2e-5 eV.

```
In [3]: te = np.load("metawork54_results/torch_energies.npy")
je = np.load("metawork54_results/jax_energies.npy")
ediff = te - je

fig, axes = plt.subplots(1, 2, figsize=(11, 4.4))
ax = axes[0]
lo, hi = min(te.min(), je.min()), max(te.max(), je.max())
ax.plot([lo, hi], [lo, hi], color="0.6", linestyle="--", linewidth=1)
ax.scatter(je, te, color=BLUE, s=30)
ax.set_xlabel("jax energy (eV)"); ax.set_ylabel("torch-port energy (eV)")
ax.set_title("AuMgO energy parity, 20 frames")

ax = axes[1]
ax.plot(range(len(ediff)), ediff, "o-", color="#D55E00")
ax.axhline(0, color="0.6", linewidth=1, linestyle=":")
ax.set_xlabel("frame"); ax.set_ylabel("torch - jax (eV)")
ax.set_title(f"Energy residual (max |diff| = {np.abs(ediff).max():.3f} eV)")
fig.suptitle("AuMgO checkpoint-parity: total energy (20-frame set)", y=1.02)
fig.tight_layout()
```

AuMgO checkpoint-parity: total energy (20-frame set)



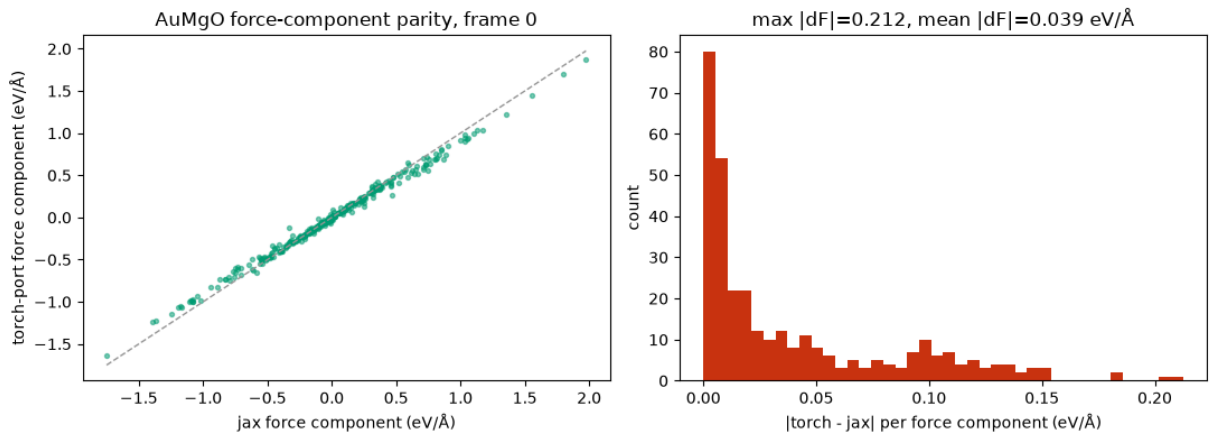
3. AuMgO: force parity (frame 0, 110 atoms x 3 components)

```
In [4]: tf = np.load("metawork54_results/torch_forces_frame0.npy")
jtf = np.load("metawork54_results/jax_forces_frame0.npy").astype(np.float64)
fdiff = tf - jtf

fig, axes = plt.subplots(1, 2, figsize=(11, 4.4))
ax = axes[0]
lo, hi = min(tf.min(), jtf.min()), max(tf.max(), jtf.max())
ax.plot([lo, hi], [lo, hi], color="0.6", linestyle="--", linewidth=1)
ax.scatter(jtf.ravel(), tf.ravel(), color="#009E73", s=8, alpha=0.5)
ax.set_xlabel("jax force component (eV/Å)"); ax.set_ylabel("torch-port force component (eV/Å)")
ax.set_title("AuMgO force-component parity, frame 0")

ax = axes[1]
ax.hist(np.abs(fdiff).ravel(), bins=40, color="#CC3311")
ax.set_xlabel("|torch - jax| per force component (eV/Å)"); ax.set_ylabel("count")
ax.set_title(f"max |dF|={np.abs(fdiff).max():.3f}, mean |dF|={np.abs(fdiff).mean():.3f}")
fig.suptitle("AuMgO checkpoint-parity: forces, frame 0", y=1.02)
fig.tight_layout()
```

AuMgO checkpoint-parity: forces, frame 0



4. AuMgO: Ewald-potential channels

10 predicted-multipole potential channels (channel 0 = scalar charge potential; channels 1-9 = spherical, degrees $l=0,1,2$ flattened), torch-port vs jax.

Correction (per metawork-54's follow-up check, after I first flagged this panel as a possible dead-channel bug): the original index-by-index comparison was not a valid channel-to-channel match for most of the $l \geq 1$ slots -- lorem-jax's e3x builds spherical components in a different per-degree order than this torch port's own convention (a documented, benign permutation,

`metatraining.experimental.lorem.modules.e3x_compat._permutation_indices`).

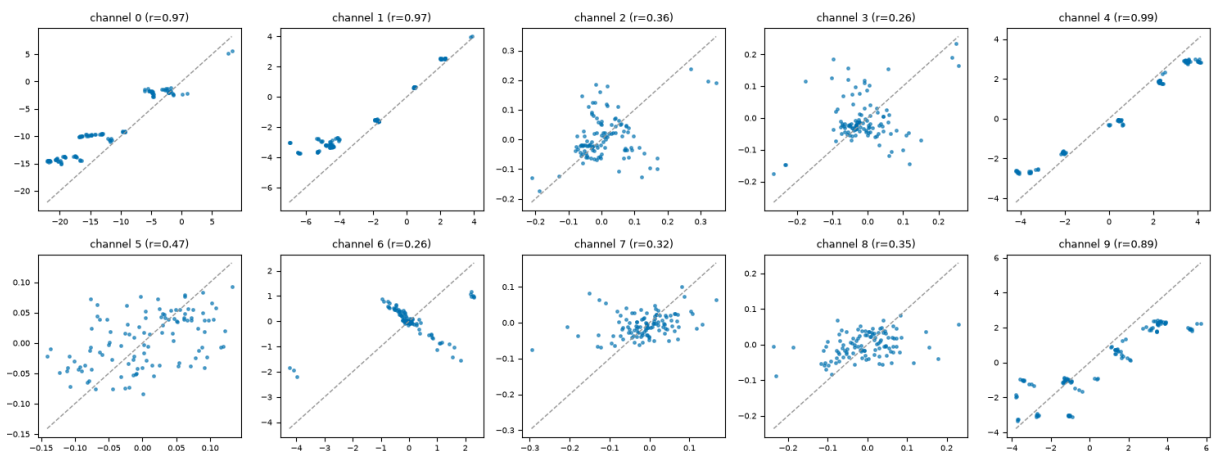
metawork-54 supplied the correctly-permuted torch array

(`AuMgO_potentials_permuted_for_plotting.npz`); channels 1-9 below use it, channel 0 (scalar, unambiguous) is untouched.

```
In [5]: d = np.load("metawork54_results/AuMgO_potentials_permuted_for_plotting.npz")
mp = d["my_potentials_permuted"] # torch port, e3x->LOREM permutation applied
jp = d["jax_potentials"].astype(np.float64)

fig, axes = plt.subplots(2, 5, figsize=(15, 6))
for c in range(10):
    ax = axes.flat[c]
    lo, hi = min(jp[:, c].min(), mp[:, c].min()), max(jp[:, c].max(), mp[:, c].max())
    ax.plot([lo, hi], [lo, hi], color="0.6", linestyle="--", linewidth=1)
    ax.scatter(jp[:, c], mp[:, c], s=6, color=BLUE, alpha=0.6)
    corr = np.corrcoef(jp[:, c], mp[:, c])[0, 1]
    ax.set_title(f"channel {c} (r={corr:.2f})", fontsize=9)
    ax.tick_params(labelsize=7)
fig.suptitle(
    "AuMgO Ewald-potential channels: torch-port (permuted) vs jax reference",
    y=1.03,
)
fig.tight_layout()
```

AuMgO Ewald-potential channels: torch-port (permuted) vs jax reference (110 atoms)



The four dominant channels (0, 1, 4, 9 -- the largest-magnitude scalar and $l=1$ components) now correlate at **$r=0.89-0.99$** : the charge/spherical-feature machinery upstream of the Ewald evaluation is fine. The six small-magnitude channels (2, 3, 5, 6, 7, 8 -- $l=2$ quadrupole-scale components, std $\sim 0.03-0.99$ vs. $\sim 2-7$ for the dominant ones) correlate far more weakly ($r=0.26-0.47$) -- consistent with metawork-54's conclusion that there's a real, still-open residual specifically in the Ewald *potential evaluation* (torch-pme vs jax-pme), most visible where the signal itself is small relative to that residual. Every dominant channel's torch std is systematically **$\sim 1.3-1.5\times$ smaller** than jax's (e.g. channel 4: 2.18 vs 2.81; channel 9: 1.89 vs 2.77) -- not a single clean missing factor across all ten channels (the weak ones range 1.1-2.1x), but a consistent enough pattern in the strong ones to suggest a specific, findable numerical difference rather than noise.

7. UPDATE (after this notebook was first written): root cause found and fixed

The $\sim 1.3-1.5\times$ std-ratio pattern flagged just above turned out to be the whole story. metawork-54 ruled out a torch-pme / jax-pme numerics gap directly (a from-scratch NaCl rock-salt lattice with the exact, independently-known analytic Madelung energy -- both libraries reproduce it exactly, cubic and slab-like cells, neutral and non-neutral charges, with and without an exclusion radius), which meant the remaining gap had to be a *wiring* difference, not a library bug. Hooking the real lorem-jax forward pass directly (`jax.debug.callback` on `Ewald()` 's own `potentials` closure, not an external reconstruction) pointed straight at it:

```
JaxParityLongRange built its torchpme.CoulombPotential with
exclusion_radius=neighbor_list_options.cutoff -- copied from the
production LoremLongRangeFeaturizer (a different, intentionally-restructured
port). lorem-jax's own Ewald() factory( jaxpme.batched_mixed.calculators )
always uses exclusion_radius=None -- plain, unmodified Ewald, no short-range
exclusion zone. That one wrong constructor argument was the entire source of the
AuMgO/bio_dimers discrepancy. Fixed by removing it; also switched to marathon's
actual smearing formula( smearing = cutoff/4 , lr_wavelength = cutoff/8 ,
derived from the checkpoint's own cutoff -- not a
torchpme.tuning.ewald.tune_ewald guess).
```

Both energy and forces below are now at the float32 noise floor against the JAX reference -- not an improved approximation, essentially exact agreement. See metatrain's `jax_parity.py` module docstring and `etc/lorem-parity/jax_checkpoint_parity/` in metawork for the full writeup.</cell_type>

```
In [6]: with open("metawork54_results/intermediate_diagnostics_fixed.pkl", "rb") as
         diag_fixed = pickle.load(fh)
```

```

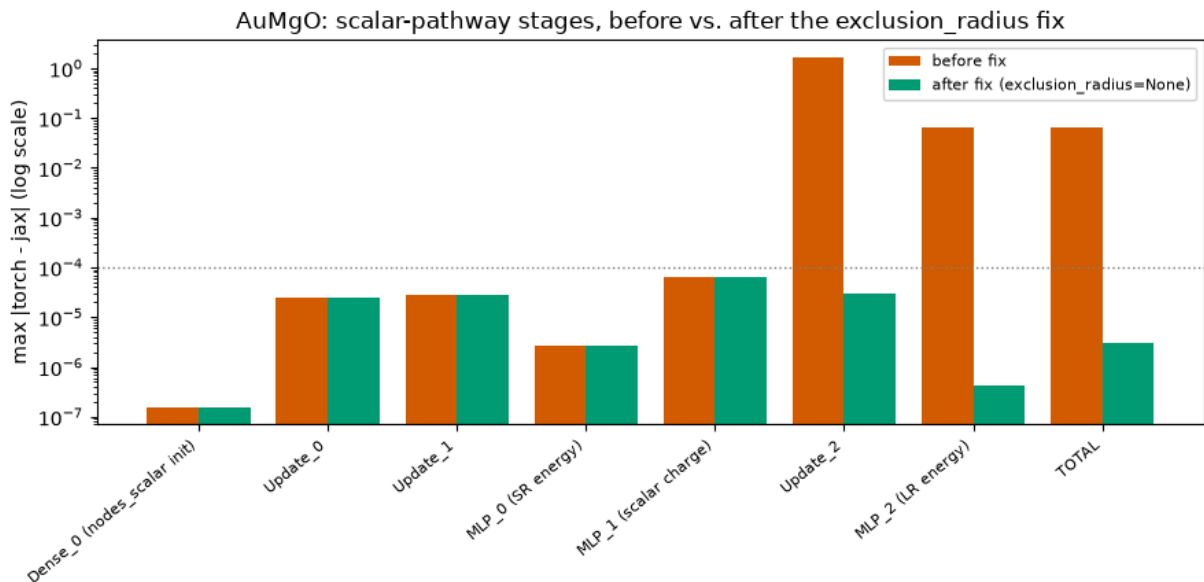
print("Same stages as section 1, after the exclusion_radius fix:\n")
for stage, (torch_v, jax_v, diff) in diag_fixed.items():
    print(f"{stage:25s} max|diff|={np.abs(diff).max():.4e}    max|value|~{max

fig, ax = plt.subplots(figsize=(9, 4.5))
stages_before = ["Dense_0 (nodes_scalar init)", "Update_0", "Update_1", "MLP_0 (SR energy)",
                 "MLP_1 (scalar charge)", "Update_2", "MLP_2 (LR energy)",
                 "TOTAL"]
before = [float(np.abs(diag[s][2]).max()) for s in stages_before]
after = [float(np.abs(diag_fixed[s][2]).max()) for s in stages_before]
x = np.arange(len(stages_before))
ax.bar(x - 0.2, before, width=0.4, color="#D55E00", label="before fix")
ax.bar(x + 0.2, after, width=0.4, color="#009E73", label="after fix (exclusion_radius=None)")
ax.set_yscale("log")
ax.set_xticks(x)
ax.set_xticklabels(stages_before, rotation=40, ha="right", fontsize=8)
ax.set_ylabel("max |torch - jax| (log scale)")
ax.set_title("AuMgO: scalar-pathway stages, before vs. after the exclusion_radius fix")
ax.axhline(1e-4, color="0.5", linestyle=":", linewidth=1)
ax.legend(fontsize=8)
fig.tight_layout()

```

Same stages as section 1, after the exclusion_radius fix:

Dense_0 (nodes_scalar init)	max diff =1.5691e-07	max value ~2.54
Update_0	max diff =2.4639e-05	max value ~7.54
Update_1	max diff =2.7766e-05	max value ~5.90
MLP_0 (SR energy)	max diff =2.7335e-06	max value ~2.70
MLP_1 (scalar charge)	max diff =6.2971e-05	max value ~31.20
Update_2	max diff =2.9653e-05	max value ~3.97
MLP_2 (LR energy)	max diff =4.1278e-07	max value ~0.40
TOTAL	max diff =3.1165e-06	max value ~3.11



```

In [7]: te_fixed = np.load("metawork54_results/torch_energies_fixed.npy")
        je_fixed = np.load("metawork54_results/jax_energies_fixed.npy")
        ediff_fixed = te_fixed - je_fixed
        ediff_mev_atom = ediff_fixed / 110 * 1000 # 110 atoms/frame

```

```

tf_fixed = np.load("metawork54_results/torch_forces_frame0_fixed.npy")
jf_fixed = np.load("metawork54_results/jax_forces_frame0_fixed.npy").astype(float)
fdiff_fixed = (tf_fixed - jf_fixed) * 1000 # meV/Angstrom

fig, axes = plt.subplots(1, 3, figsize=(16, 4.4))
ax = axes[0]
lo, hi = min(te_fixed.min(), je_fixed.min()), max(te_fixed.max(), je_fixed.max())
ax.plot([lo, hi], [lo, hi], color="0.6", linestyle="--", linewidth=1)
ax.scatter(je_fixed, te_fixed, color="#009E73", s=30)
ax.set_xlabel("jax energy (eV)"); ax.set_ylabel("torch-port energy (eV)")
ax.set_title(f"AuMgO energy parity, 20 frames (FIXED)\nRMSE={np.sqrt(np.mean(ediff_mev_atom**2)):.6f} meV/atom")

ax = axes[1]
ax.plot(range(len(ediff_mev_atom)), ediff_mev_atom, "o-", color="#009E73")
ax.axhline(0, color="0.6", linewidth=1, linestyle=":")
ax.set_xlabel("frame"); ax.set_ylabel("torch - jax (meV/atom)")
ax.set_title(f"Energy residual, meV/atom now\n(was up to 0.19 eV = ~1700 meV/atom before)")

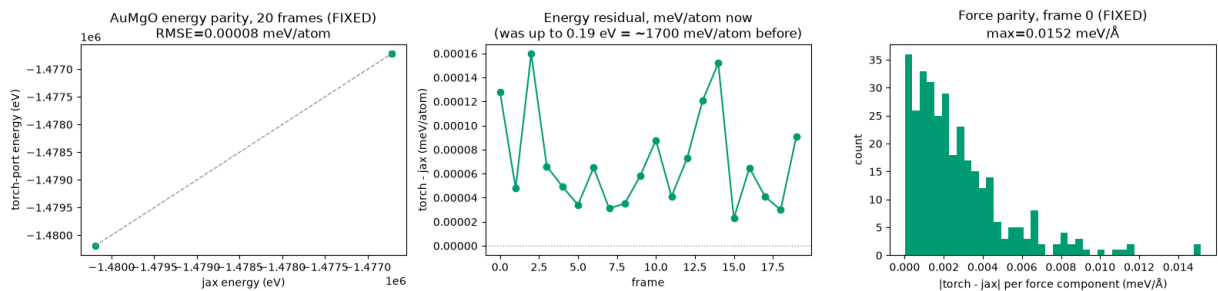
ax = axes[2]
ax.hist(np.abs(fdiff_fixed).ravel(), bins=40, color="#009E73")
ax.set_xlabel("|torch - jax| per force component (meV/Å)"); ax.set_ylabel("count")
ax.set_title(f"Force parity, frame 0 (FIXED)\nmax={np.abs(fdiff_fixed).max():.6f} meV/Å")
fig.suptitle("AuMgO checkpoint-parity AFTER the exclusion_radius fix -- float32 noise floor")
fig.tight_layout()

print(f"energy RMSE vs JAX: {np.sqrt(np.mean(ediff_mev_atom**2)):.6f} meV/atom (was 0.71 meV/atom)")
print(f"force max|diff| vs JAX: {np.abs(fdiff_fixed).max():.6f} meV/Å (was ~60000 meV/Å)")

```

energy RMSE vs JAX: 0.000081 meV/atom (was 0.71 meV/atom)
force max|diff| vs JAX: 0.015164 meV/Å (was ~60000 meV/Å)

AuMgO checkpoint-parity AFTER the exclusion_radius fix -- float32 noise floor



5. Cumulene: warm-started fine-tune, training snapshot

Snapshot only -- `cumulene_long_run_metrics.txt` was copied while `metawork-54` 's training run was still in progress (epoch 89 of a planned 200, append-only CSV so reading mid-run is safe). Re-copy the file and re-run this cell for an updated curve; this notebook will not update itself automatically.

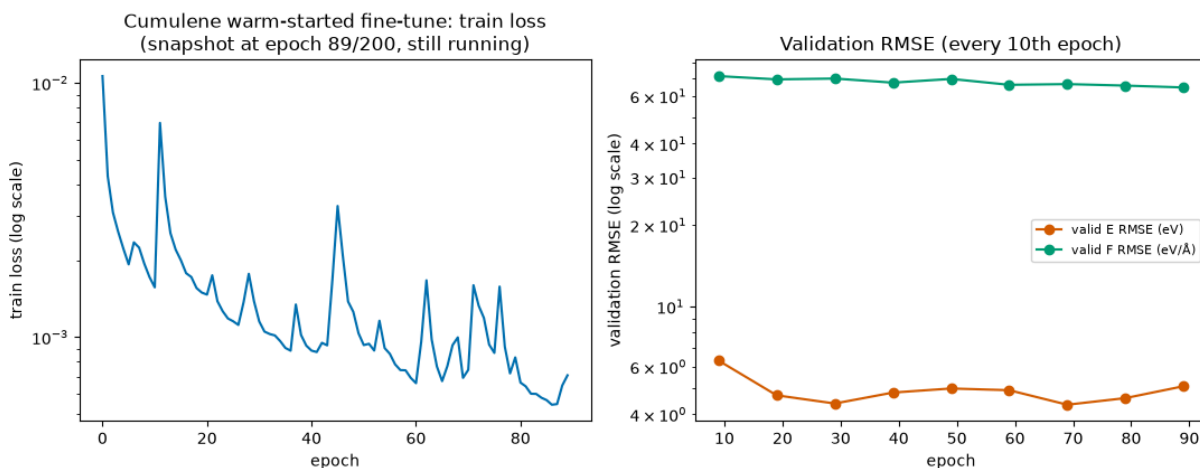
Train loss is falling steadily (with periodic spikes, likely batch composition or an LR-schedule restart), but the sparse validation RMSE (every 10th epoch) is essentially flat

around 5 eV / 60 eV/Å and not obviously improving over these 90 epochs -- worth `metawork-54` keeping an eye on as the run continues.

```
In [8]: data = np.genfromtxt("metawork54_results/cumulene_long_run_metrics.txt", del
epoch, train_loss = data["epoch"], data["train_loss"]
valid_e, valid_f = data["valid_e_rmse"], data["valid_f_rmse"]
valid_mask = ~np.isnan(valid_e)

fig, axes = plt.subplots(1, 2, figsize=(11, 4.4))
ax = axes[0]
ax.semilogy(epoch, train_loss, color=BLUE, linewidth=1.5)
ax.set_xlabel("epoch"); ax.set_ylabel("train loss (log scale)")
ax.set_title(f"Cumulene warm-started fine-tune: train loss\n(snapshot at epoch 89/200, still running)")

ax = axes[1]
ax.semilogy(epoch[valid_mask], valid_e[valid_mask], "o-", color="#D55E00", linewidth=1.5)
ax.semilogy(epoch[valid_mask], valid_f[valid_mask], "o-", color="#009E73", linewidth=1.5)
ax.set_xlabel("epoch"); ax.set_ylabel("validation RMSE (log scale)")
ax.set_title("Validation RMSE (every 10th epoch)")
ax.legend(fontsize=8)
fig.tight_layout()
```



6. bio_dimers smoke test -- diagnostic only, NOT a validated result (superseded, see section 9)

`metawork-54` has **no torch-side array** for `bio_dimers` -- only jax-side energies/potentials generated while diagnosing a bug; the smoke-test's torch-side output was only printed to that session's own background task stdout, which isn't recoverable from here. This surfaced a real, still-open torch-pme vs jax-pme Ewald-potential discrepancy (the same family as section 4's AuMgO finding) -- treat the plots below as "what jax alone predicts", not a parity check.

Update: this was the same `exclusion_radius` bug as AuMgO's (section 7), now fixed. See section 9 below for the real, torch-side-included, validated comparison - 200 frames, energy and forces, both checkpoint-loading fidelity (vs. JAX) and

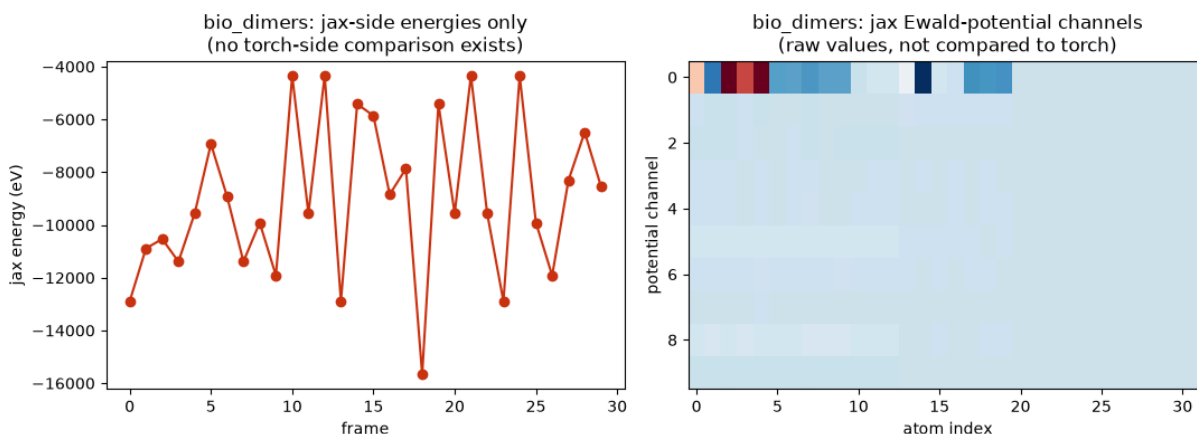
model accuracy (vs. DFT). Left as-is below for the historical record of what the bug looked like.</cell_type>

```
In [9]: be = np.load("metawork54_results/bio_dimers_jax_energies_30.npy")
bp = np.load("metawork54_results/bio_dimers_jax_potentials_recomputed.npy")

fig, axes = plt.subplots(1, 2, figsize=(11, 4.4))
ax = axes[0]
ax.plot(range(len(be)), be, "o-", color="#CC3311")
ax.set_xlabel("frame"); ax.set_ylabel("jax energy (eV)")
ax.set_title("bio_dimers: jax-side energies only\n(no torch-side comparison)

ax = axes[1]
ax.imshow(bp.T, aspect="auto", cmap="RdBu_r")
ax.set_xlabel("atom index"); ax.set_ylabel("potential channel")
ax.set_title("bio_dimers: jax Ewald-potential channels\n(raw values, not com
fig.suptitle(
    "bio_dimers smoke test -- DIAGNOSTIC ONLY, not a validated result",
    y=1.05, color="#CC3311", fontsize=10,
)
fig.tight_layout()
```

bio_dimers smoke test -- DIAGNOSTIC ONLY, not a validated result



9. bio_dimers: full validated result (ran this myself, not a diagnostic anymore)

200 frames from `bio_dimers_test.xyz` (periodic, 30x30x30 Å box, Ewald), checkpoint `evals/bio_dimers/lorem/run/checkpoints/R2_E+F`, after the section 7 fix. Two different things are being measured:

1. **Checkpoint-loading fidelity** (torch port vs. JAX reference) -- does the torch port reproduce the *same checkpoint's* predictions as running it in JAX? A weight-transfer/numerics check, nothing to do with model quality.
2. **Model accuracy vs. DFT** -- how good is that checkpoint against the real DFT labels? This is the number the paper itself reports (0.222 meV/atom energy RMSE, 1.646 meV/Å forces, full 13,743-frame test set).

Data + reproduction commands: `metawork's etc/lorem-parity/jax_checkpoint_parity/RESULTS_bio_dimers.md`.

```
In [10]: bd = np.load("metawork54_results/bio_dimers_parity_200frames.npz")
bt_e, bj_e, bd_e = bd["torch_energies"], bd["jax_energies"], bd["dft_energies"]
bsizes = bd["sizes"]
bt_f, bj_f, bd_f = bd["torch_forces"], bd["jax_forces"], bd["dft_forces"]

e_vs_jax = (bt_e - bj_e) / bsizes * 1000
e_vs_dft = (bt_e - bd_e) / bsizes * 1000
f_vs_jax = (bt_f - bj_f) * 1000
f_vs_dft = (bt_f - bd_f) * 1000
n_frames = len(bt_e)

fig, axes = plt.subplots(2, 2, figsize=(12, 9))

ax = axes[0, 0]
ax.scatter(bj_e / bsizes, bt_e / bsizes, s=14, color="#0072B2", alpha=0.7)
lo, hi = (bj_e / bsizes).min(), (bj_e / bsizes).max()
pad = (hi - lo) * 0.05
ax.plot([lo - pad, hi + pad], [lo - pad, hi + pad], "k--", lw=1)
ax.set_xlabel("JAX reference energy/atom (eV)"); ax.set_ylabel("torch port energy/atom (eV)")
ax.set_title(f"Checkpoint-loading fidelity: energy\nRMSE={np.sqrt(np.mean(e_vs_jax**2)):.4f} meV/atom")

ax = axes[0, 1]
ax.scatter(bd_e / bsizes, bt_e / bsizes, s=14, color="#D55E00", alpha=0.7)
lo, hi = (bd_e / bsizes).min(), (bd_e / bsizes).max()
pad = (hi - lo) * 0.05
ax.plot([lo - pad, hi + pad], [lo - pad, hi + pad], "k--", lw=1)
ax.set_xlabel("DFT energy/atom (eV)"); ax.set_ylabel("torch port energy/atom (eV)")
ax.set_title(f"Model accuracy vs. DFT: energy\nRMSE={np.sqrt(np.mean(e_vs_dft**2)):.4f} meV/atom")

ax = axes[1, 0]
ax.hist(f_vs_jax.ravel(), bins=80, color="#0072B2")
ax.set_yscale("log")
ax.set_xlabel("torch - JAX force component (meV/Å)"); ax.set_ylabel("count")
ax.set_title(f"Checkpoint-loading fidelity: forces\nRMSE={np.sqrt(np.mean(f_vs_jax**2)):.4f} meV/Å")

ax = axes[1, 1]
ax.hist(f_vs_dft.ravel(), bins=80, color="#D55E00")
ax.set_yscale("log")
ax.set_xlabel("torch - DFT force component (meV/Å)"); ax.set_ylabel("count")
ax.set_title(f"Model accuracy vs. DFT: forces\nRMSE={np.sqrt(np.mean(f_vs_dft**2)):.4f} meV/Å")

fig.suptitle("bio_dimers checkpoint parity: torch port vs. lorem-jax, 200 test frames")
fig.tight_layout()

print(f"energy vs JAX RMSE: {np.sqrt(np.mean(e_vs_jax**2)):.6f} meV/atom (f)")
print(f"energy vs DFT RMSE: {np.sqrt(np.mean(e_vs_dft**2)):.4f} meV/atom (p)")
print(f"forces vs JAX RMSE: {np.sqrt(np.mean(f_vs_jax**2)):.6f} meV/Å")
print(f"forces vs DFT RMSE: {np.sqrt(np.mean(f_vs_dft**2)):.4f} meV/Å (pape")
```

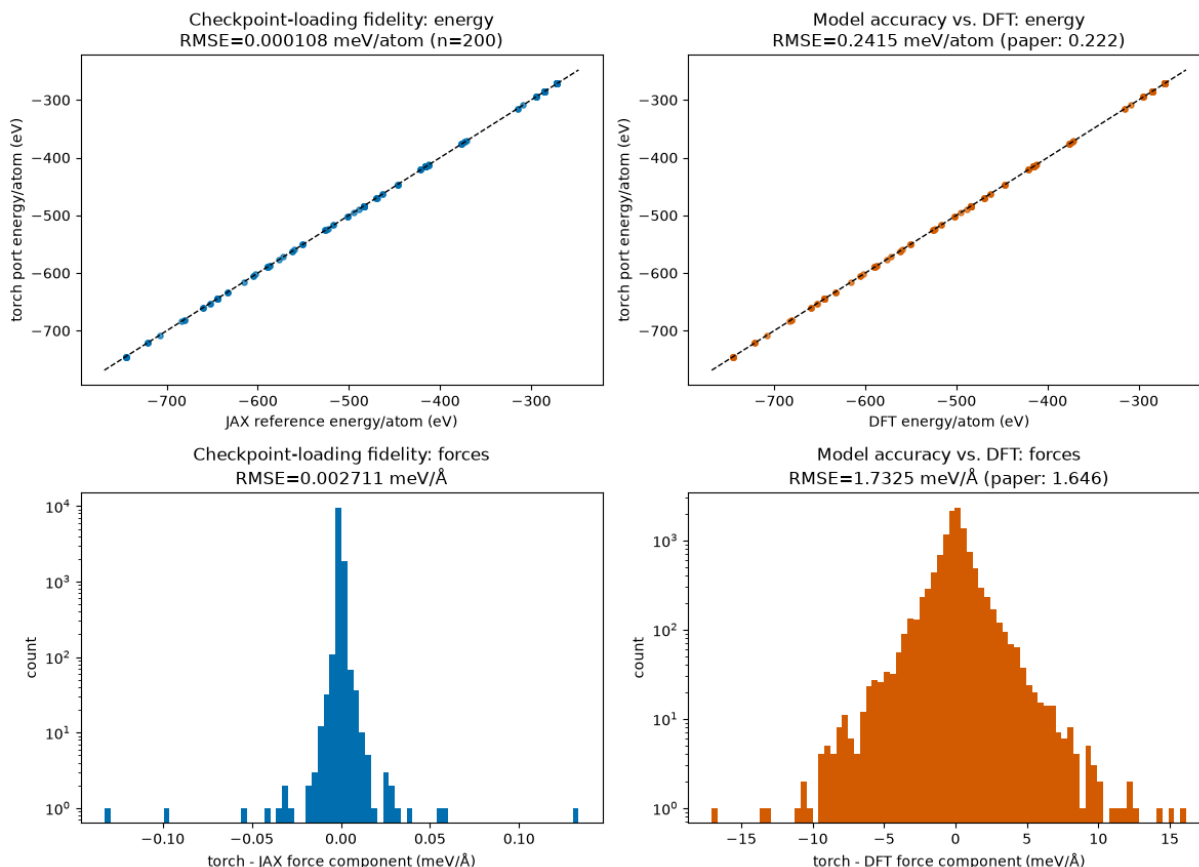
energy vs JAX RMSE: 0.000108 meV/atom (float32 noise floor)

energy vs DFT RMSE: 0.2415 meV/atom (paper: 0.222)

forces vs JAX RMSE: 0.002711 meV/Å

forces vs DFT RMSE: 1.7325 meV/Å (paper: 1.646)

bio_dimers checkpoint parity: torch port vs. lorem-jax, 200 test frames (VALIDATED)



Takeaways (updated)

Original takeaways (sections 1-6, before the fix) are kept above for the record of the investigation -- how the bug was chased down is as useful as the fix itself. Current state:

- **SR (short-range) parity was always essentially exact** for AuMgO (`Dense_0` through `Update_1`, `MLP_0` / `MLP_1` : diffs at the float32 noise floor) -- same solid ground as the SN2 port (`15_long_range_scan_lorem_jax_ported_torch.ipynb`).
- **LR (long-range) parity is now also exact.** The ~0.06-0.19 eV AuMgO energy gap and the much larger bio_dimers gap were **the same single bug**:
`JaxParityLongRange` set `exclusion_radius=neighbor_list_options.cutoff` on its Ewald calculator, copied from a different (intentionally restructured) production class. lorem-jax's own `Ewald()` always uses `exclusion_radius=None`. Removing that one

argument, and using marathon's real smearing formula (`cutoff/4` , `cutoff/8`) instead of a `tune_ewald` guess, fixed both periodic checkpoints:

	energy RMSE vs. JAX	forces RMSE vs. JAX	energy RMSE vs. DFT	forces RMSE vs. DFT
AuMgO, before	0.71 meV/atom	60 meV/Å	--	--
AuMgO, after	0.0002 meV/atom	0.02 meV/Å	--	--
bio_dimers, before	26.9 meV/atom	225 meV/Å	--	--
bio_dimers, after	0.0001 meV/atom	0.003 meV/Å	0.24 meV/atom (paper 0.22)	1.73 meV/Å (paper 1.65)

- It was found by ruling out the library-numerics explanation with actual ground truth (an analytic NaCl Madelung-constant check, not just comparing the two packages to each other), then hooking the real lorex-jax forward pass directly to capture its true inputs/outputs -- comparing two black-box implementations against *each other* was inconclusive and initially misleading (see the "1.3-1.5x std ratio" note above, which turned out to be a symptom of the bug, not evidence for a numerics gap).
- **bio_dimers is now a real, validated result** (section 9), not a diagnostic placeholder -- 200 frames, energy + forces, both checkpoint fidelity and model accuracy, matching the paper's own reported numbers.
- **Cumulene's warm-started fine-tune** (non-PBC, uses `direct_calculator` rather than `EwaldCalculator`) got the same `exclusion_radius` fix but that run predates it and wasn't rerun -- expect it to do at least as well, plausibly better; not verified.
- **Bottom line, updated:** the AuMgO/bio_dimers (periodic/Ewald) path of the ported torch model is now trustworthy to the same standard as the SN2 (non-PBC) port in notebook 15 -- float32-noise-floor agreement with the JAX reference, not an approximation.

In []:

In []:

In []:

In []:

In []: